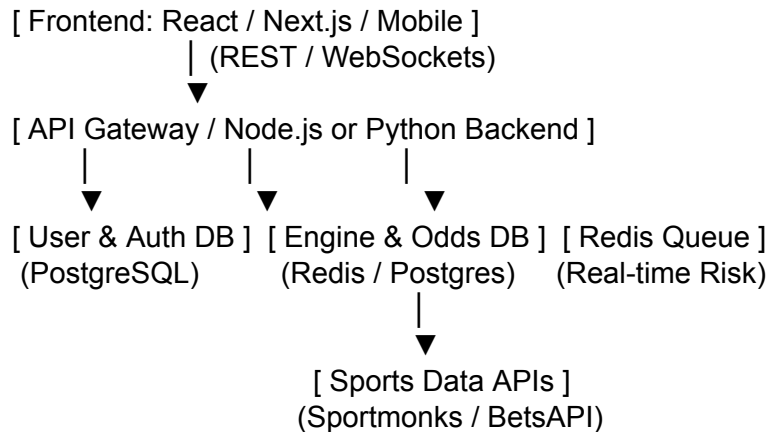


Here is the end-to-end technical developer flow to build an automated sports prop firm platform.

Technical Architecture Overview



Step-by-Step Developer Implementation

1. Schema & Database Setup

Phase 1: Foundation

Design the relational database (PostgreSQL/MySQL) around three main core domains:

Users & Subscriptions: Accounts, KYC status, and purchased challenge packages.

Challenge Instances: Virtual initial bankroll, current virtual balance, peak high-water mark equity, daily starting equity, and challenge status (active, passed, failed). Picks/Tickets &

Odds: User selections, market type, locked odds at timestamp, stake, potential payout, and ticket status (pending, won, lost, void).

2. Sports Data & Live Odds Ingestion

Phase 2: Data Pipeline

Connect to a reliable sports data API (such as Sportmonks or BetsAPI): Fixtures & Odds

Sync: Set up a background worker (using Celery or BullMQ) to fetch upcoming football matches, player prop markets (e.g., goals, cards, shots on target), and live odds

movement. Cache Layer: Store live odds in Redis with a short TTL (5–15 seconds) so your web app can serve high-concurrency traffic without exceeding API rate limits.

3. Pick Execution & Validation Engine

Phase 3: Core Business Logic

When a user submits a prop pick from the client interface: Pre-Trade Risk Check: Verify that the user has an active challenge status, the stake does not exceed the maximum

allowed per match (e.g., max 2% of bankroll), and the match has not kicked off. Odds

Locking: Compare the submitted odds against the cached odds in Redis to prevent slippage or latency exploitation. Ledger Update: Deduct the virtual stake from

current_balance and write the ticket to the database inside a database transaction.

4. Real-Time Risk & Drawdown Engine

Phase 4: Risk Enforcer

Run an asynchronous, real-time cron/worker script that continuously monitors open equity

across all accounts: Daily Loss Limit: Every midnight (UTC or local time), snapshot the

starting equity. If `current_equity` drops below `starting_equity * (1 - daily_loss_pct)` at any point during the day, mark the account as failed. Max Total Drawdown: Track the account's historical peak equity (high-water mark). If equity drops below `initial_bankroll * (1 - max_drawdown_pct)` [or trailing peak], immediately set account status to failed and cancel open pending picks.

5. **Automated Settlement Engine**

Phase 5: Event Processing

Listen for match completion webhooks or poll the sports API for finished fixture results:

Grade Picks: Parse match statistics (e.g., individual player goal counts, team corner

totals) against pending bet conditions. Mark tickets as won or lost. Credit Virtual Payouts:

Update `current_balance` and recalculate total return on investment (ROI). Evaluation

Checker: Evaluate if `current_balance` has met or exceeded the profit target (e.g., +10%).

If passed and no risk limits were breached, update the user account to funded and trigger an automated congratulatory email/notification.

6. **Payments, Payouts & Admin Portal**

Phase 6: Monetization

Inbound Payment Gateways: Integrate local and international payment rails (Paystack,

Flutterwave, or Crypto) to handle challenge purchase checkouts. Payout Disbursement:

Build a request portal where funded users can request profit withdrawals based on their

split (e.g., 80/20). Integrate payout APIs to disburse funds automatically upon manual

admin approval.